

AI Engineering Summer Training Daily Activity Log

Student Name: Mohammed Bin Jamil

Student ID: 2339194

Week: 2

Supervisor: Eng: Abdullah Barghash

Instructions

Complete this document at the end of every training day. At the end of each week, upload the updated document to the shared Google Drive folder.

When you upload the file to drive rename from `AI_Engineering_Daily_Project_Log_Template.docx` to `FirstName_ID_WeekNum_docx`

Day 1 Date: Jul/12/2026 Today's Topic: Interactive Frontend Architecture and Focus Engine Setup

Daily Task Completed:

- Initiated the development of the frontend Focus Budget Visualizer application.
- Designed the core state engine using vanilla JavaScript to process focal states and their distinct energy depletion/regeneration profiles.
- Setup and ran a local development environment utilizing a Python HTTP server (python -m http.server 8000) to test files locally.

What I Learned (3-5 points):

- I learned how to build custom state-machine logic in JavaScript to cycle through specific user modes.
- I realized how native timing intervals such as setInterval can govern recurring real-time tick cycles.
- I understood how to host and test frontend components locally using python-based server utilities.
- I learned how state-driven math calculations dictate system performance, keeping state-processing functions separate from display scripts.

Work Evidence (GitHub / Colab / Files):

<https://github.com/MkJamil/O-LLM/blob/main/Submission/Vibecoding-spec.pdf>

Challenges Faced:

- It was difficult to keep the UI synchronized in real-time with the ticking state changes, which initially led to laggy and missed value updates.

How I Solved Them:

- Decoupled the computational core state loop from visual rendering by creating a separate display handler function that re-renders only when a change is detected.

Self-Evaluation (Excellent / Good / Fair / Need More Practice):

Good

Supervisor Comments:

Day 2 Date: Jul/13/2026 Today's Topic: Responsive UI Grid Design and Dynamic SVG Gauges

Daily Task Completed:

- Implemented a responsive three-column grid layout using Tailwind CSS classes.
 - Structured and integrated an interactive, high-fidelity SVG circular ring gauge with center-aligned percentage texts.
 - Positioned horizontal state-switching action selectors for interactive user control.
-

What I Learned (3–5 points):

- I learned to build robust multi-column dashboard layouts using Tailwind grid layouts such as grid-cols-1 md:grid-cols-3.
- I discovered how SVG stroke parameters, specifically stroke-dasharray and stroke-dashoffset, are mapped to represent numerical percentages.
- I learned how to leverage flexbox positioning rules to vertically center interactive gauges and controls inside empty layout cards.

Work Evidence (GitHub / Colab / Files):

https://github.com/MkJamil/Focus_Budget

Challenges Faced:

- Experienced issues with vertical alignment where the center column cards stretched awkwardly, creating massive white space above the footer.
-

How I Solved Them:

- Consolidated the circular gauge wrapper and its selector buttons into a single flex container and removed extraneous height limits.
-

Self-Evaluation:

Good

Supervisor Comments:

Day 3 Date: Jul/14/2026 Today's Topic: HTML5 Canvas Visualizations and Data Persistence

Daily Task Completed:

- Programmed a real-time "Energy Trend" line chart and a vertical "Dynamic Energy Bar" graph using HTML5 Canvas APIs.
- Integrated local storage modules to dynamically track, save, and retrieve gamified "Focus Streak" records.

What I Learned (3-5 points):

- I learned the procedural canvas commands needed to draw lines, clear drawing frames, and plot dynamic coordinates.
- I discovered the functional differences between vector SVG components and pixel-based Canvas environments.
- I mastered using browser localStorage utilities to read and write key-value pairs, letting user states survive page refreshes.

Work Evidence (GitHub / Colab / Files):

https://github.com/MkIamil/Focus_Budget

Challenges Faced:

- Encountered a rendering bug where the line chart's interactive coordinate circle drifted past the container and floated over surrounding layout boxes.

How I Solved Them:

- Programmed strict boundary clamping controls using math utilities to lock tracking coordinates within the graph bounds and added visual CSS clip-path parameters to the canvas element.

Self-Evaluation (Excellent / Good / Fair / Need More Practice):

Excellent

Supervisor Comments:

Day 4 Date: Jul/15/2026 Today's Topic: Immersive UI States and Atmospheric Styling

Daily Task Completed:

- Rearranged layout cards (moving the Audit Ledger and Time Distribution logs to the left side and overrides to the right) to improve dashboard utility.
- Added animated, energy-reactive radial gradient glows that shift colors dynamically behind visual containers.
- Integrated dynamic color-wash transitions that smoothly change the site's atmosphere upon state modifications.

What I Learned (3-5 points):

- I learned to build interactive, state-reactive layout components that transition smoothly over a 0.8s window.
- I learned how to combine radial CSS gradients with CSS backdrop blur settings to create modern dashboard depth.
- I discovered how to manage multiple CSS theme configurations (Classic Dark, Sage Oasis, and Cyberpunk Minimalist) within a single style manifest.

Work Evidence (GitHub / Colab / Files):

<https://github.com/MkJamil/O-LLM/blob/main/Submission/Focus%20Budget%20-%20Bug%20Log.pdf>

Challenges Faced:

- Discovered visual clutter and performance lags caused by redundant, duplicate theme selectors rendering simultaneously in the header and the sidebar.

How I Solved Them:

- Purged the duplicate header dropdown menu, bound all global style rules to the left-hand theme card, and consolidated selector events to resolve layout issues.

Self-Evaluation (Excellent / Good / Fair / Need More Practice):

Good

Supervisor Comments:

Day 5 Date: Jul/16/2026 Today's Topic: Final Assembly, Code Refactoring, and Quality Assurance

Daily Task Completed:

- Conducted visual and semantic audits to rename the leftover "Admin" tracking actions to "Email" across all scripts and menus.
- Finalized the fully responsive single-file production template of the interactive web app.
- Pushed all revised project assets to GitHub and compiled the development reflection.

What I Learned (3-5 points):

- I understood how human oversight and visual QA resolve subtle rendering issues that AI generators miss.
- I realized that structured documentation and clean variable naming are vital for shipping production-ready software.
- I learned how to coordinate development efforts between AI models and manual verification to accelerate visual web engineering.

Work Evidence (GitHub / Colab / Files):

https://github.com/MkIamil/Focus_Budget

Challenges Faced:

- Submitting massive single-file HTML code blocks to the AI generator caused prompt timeouts and server connection errors.

How I Solved Them:

- Broke modifications down into targeted, modular prompts and refactored localized blocks of code to bypass the model's processing limits.

Self-Evaluation (Excellent / Good / Fair / Need More Practice):

Good

Supervisor Comments:

Weekly Reflection

What was the most valuable thing you learned this week?

The most valuable lesson this week was mastering responsive frontend data visualization. Learning how to translate ticking mathematical state models (like depleting values by -1.8 pts during meetings or regenerating them by $+0.5\text{ pts}$ during rest) into interactive canvas graphs and dynamic SVG progress rings was an eye-opener. It bridged the gap between pure backend logical engines and user-friendly interface designs.

What was the biggest challenge?

The biggest challenge was resolving coordinate rendering overflows and vertical page alignment errors. Finding out why canvas points were floating outside their boundaries required deep debugging of visual coordinate spaces, mathematical limits, and canvas bounds tracking.

Which topic do you still need to practice?

I still need to practice managing advanced HTML5 canvas rendering optimizations. While standard redraw commands run well, understanding how to construct high-frequency canvas animations and multi-layer rendering without performance drops is a skill I look forward to exploring further.

Additional Notes

Building the Focus Budget Visualizer proved how powerful simple, native web tools can be. Using vanilla JavaScript and Tailwind CSS without heavy external frameworks like React makes the frontend lightweight, fast to load, and easy to deploy.

Weekly Submission Checklist

- ✓ Daily log completed
- ✓ Code uploaded
- ✓ GitHub updated
- ✓ Required files included
- ✓ Weekly reflection completed
- ✓ Uploaded to Google Drive